深圳大学 SHENZHEN UNIVERSITY

课程 COURSE

Big Data Analytics
大数据分析 [2026 国际商务]

Lecture 13 — Big Data in Finance 第十三讲·金融中的大数据
资产、时序、组合与风控

任课教师：戴谢尔 (Xieer Dai, David)
深圳大学 Shenzhen University

·第 3–14 周 · 星期四 11-12 节
上课地点：四方楼南 209



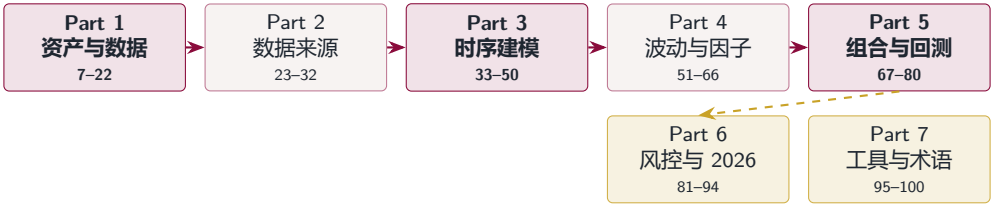
学习目标 Learning Objectives

完成本讲后，你应能够：

1. 说清四类资产与它们的收益—风险特征；
2. 读写ACF/PACF，给时间序列定阶并做ARIMA诊断；
3. 用GARCH估波动率，用协整找配对；
4. 做组合优化与诚实回测，并解释deflated Sharpe；
5. 做违约预测、评分卡与图反欺诈；
6. 说清 RAG 财报解析与模型治理的要点。

来源：对应课程大纲第 13 讲目标：了解金融数据类型与来源；掌握时序建模与文本/另类数据应用；认识回测陷阱与模型治理。

本讲路线图 Roadmap（100 帧）



主线：资产 → 数据 → 建模 → 组合 → 交易 → 风控

代码页 10 帧•样本页 16 帧；数据全合成，术语中英对照

与前面讲次的衔接 From Toolkit to Finance

讲次	给了什么	在本讲变成什么
L5–L7	回归、分类、变量选择	因子模型、违约预测（帧 65、85）
L8	降维与聚类	因子提取与风险分群（帧 64）
L9 / L10	GNN 与文本	反欺诈图、财报 RAG（帧 88、90）
L14	DID 与事件研究	事件研究法算异常收益（帧 78）

金融不是新方法，而是同一套方法在极低信噪比、非平稳、代价高昂的环境里使用。本讲每帧都强调这三件事。

来源：衔接为本讲组织。

本讲输出物 Deliverables

1) 数据源清单

交易所、Wind/Bloomberg、财报与披露、另类数据；成本与许可。

2) 11 个程序

数据、ARIMA、ADF、GARCH、协整、组合、回测、信用、反欺诈、deflated Sharpe、术语。

3) 术语表

82 条中英对照（CSV + Word/Excel），按六个主题分组。

所有数字都来自可复现的程序：`website/code/lecture13/`，种子固定 20270220；数据全部合成 (20 只股票 × 2,000 日、500 天收益率曲线、期权链、900 节点欺诈图、120 份年报文本)。

来源：程序与数据随课程仓库发布。

全讲导览 Map of the Lecture

部分	帧	关键问题
金融资产与数据类型	7–22	不同资产的收益分布长什么样？
数据来源与获取	23–32	数据从哪来？干不干净？
时间序列建模	33–50	怎么建模、怎么诊断？
波动率、状态与因子	51–66	风险怎么随时间变？
组合、交易与回测	67–80	预测怎么变成仓位？回测可不可信？
文本、信用与 2026	81–94	文本、违约、欺诈与治理
工具、术语与结语	95–100	用什么工具？术语怎么背？

读法：每个方法都走一遍数据 → 模型 → 诊断 → 应用。

来源：导览为本讲安排。

金融数据的四个特征 Four Properties

信噪比低	非平稳	厚尾	过拟合代价高
日收益的可预测部分常不到 0.1%——$R^2 < 1\%$ 是常态。	制度、政策、参与者都在变；同一个模型明年可能就不成立。	极端值远多于正态；一次黑天鹅能吃掉十年的收益。	回测漂亮、实盘归零——不是学术问题，是真金白银。

这四点决定了本讲的基调：先诊断、再建模；先基线、再复杂模型；先成本、再谈收益。

来源：四个特征据金融计量实践（Campbell, Lo & MacKinlay 1997）；表述为本讲组织。

资产类别地图 The Asset Map

类别	现金流	主要风险	数据形态
固定收益	票息 + 本金	利率、信用	曲线、成交
权益	分红 + 价差	市场、行业	逐笔行情、财报
衍生品	取决于标的	杠杆、波动率	合约、期权链
另类	租金、退出收益	流动性、估值	低频、非标

资产类别决定了收益分布的形状：债券收益窄而稳、股票宽而偏、期权是非对称的。选错分布假设，后面全错。

来源：资产地图据标准金融学教材；数据形态的归纳为本讲补充。

固定收益：从贴现到收益率

价格就是现金流的现值

$$P = \sum_{t=1}^{T} \frac{C}{(1+y)^t} + \frac{F}{(1+y)^T},$$

y 就是到期收益率 YTM——让现值等于市场价格的贴现率。

为什么它是”锚”

无风险利率是所有资产定价的分母；y 变一点，所有资产的价格都要重估。

一个反直觉的地方：票息率 $\neq$ 收益率。票息是合同写的，收益率是市场定的——两者相等时债券才按面值交易。

来源：债券定价据标准固定收益教材；YTM 与票息的区别为本讲强调。

债券：久期、凸性与”跌多少”

久期

$$D = -\frac{1}{P}\frac{\partial P}{\partial y} = \frac{\sum t \cdot PV_t}{P},$$

读作”利率上升 1%，价格跌 D%”。10 年期零息债的久期就是 10。

凸性

久期本身也随利率变——利率大幅变动时，凸性让价格跌得比久期预测的少（对多头有利）。

本讲实测（帧 11 的代码）：面值 100、票息 4%、10 年期，收益率从 3% 升到 4% 时，数值久期 8.1 年，价格跌约 7.8%。

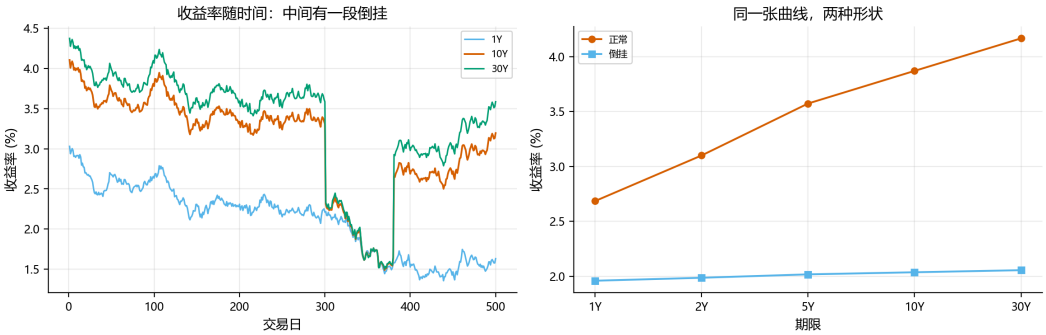
来源：久期与凸性据标准固定收益理论；实测数字来自 113_01 的债券函数。

【代码】债券定价、YTM 与数值久期

```python
def bond_price(coupon_rate, ytm, years=10, face=100.0): # price from a yield
    cash = [face * coupon_rate] * years                 # annual coupons
    cash[-1] += face                                    # the last one returns the face
    return float(sum(c / (1 + ytm) ** (t + 1) for t, c in enumerate(cash))) # discount them
def duration(coupon_rate, ytm, years=10, face=100.0): # Macaulay duration
    cash = [face * coupon_rate] * years                 # the cash flows
    cash[-1] += face                                    # plus the face value
    pv = [c / (1 + ytm) ** (t + 1) for t, c in enumerate(cash)] # present values
    weights = [p / sum(pv) for p in pv]                 # value weights
    return float(sum((t + 1) * w for t, w in enumerate(weights))) # the weighted average time
p3, p4 = bond_price(0.04, 0.03), bond_price(0.04, 0.04) # two yields
print("price at 3%%: %.2f | at 4%%: %.2f | drop %.2f%%" % (p3, p4, 100 * (p4 / p3 - 1))) # report
```

实测：票息 4%、10 年期，收益率 3% 时价格 108.5，升到 4% 时 100.0，跌 7.8%；久期约 8.1 年，与" 跌 8%" 的近似一致。完整程序见 website/code/lecture13/。

【样本 + 可视化】收益率曲线与倒挂



左：1Y/10Y/30Y 三条曲线随时间变化，中间有一段倒挂（短端高于长端）；右：正常日与倒挂日的曲线形状对比。倒挂是衰退的经典先行指标——它反映的是" 市场认为未来利率会降"。

程序 l13_01_data.py ·数据 l13_yield_curve.csv（500 天 × 5 个期限）

股票：为什么必须用总回报 Total Return

三种价格

收盘价、前复权、后复权。分红与拆股会造成"假下跌"，不复权的话回测会凭空产生亏损。

公式

$$R_t = \frac{P_t + D_t}{P_{t-1}} - 1,$$

分红再投资时用复权价；只算价差会系统性低估长期收益。

本讲的数据是收益率而不是价格，所以不存在复权问题；但你在真实项目里拿到的第一列通常是价格——第一步就是复权。

来源：总回报与复权据标准权益分析实践；提醒为本讲补充。

收益率的两种算法 Simple versus Log

简单收益

$R_t = P_t/P_{t-1} - 1$；多期不能直接相加，要连乘。

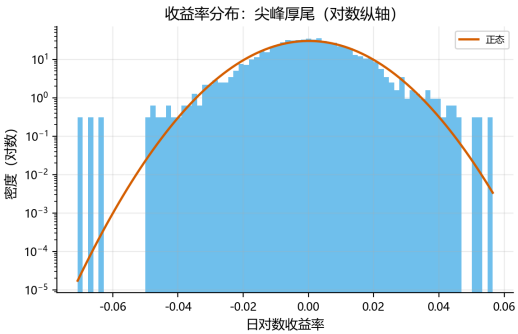
对数收益

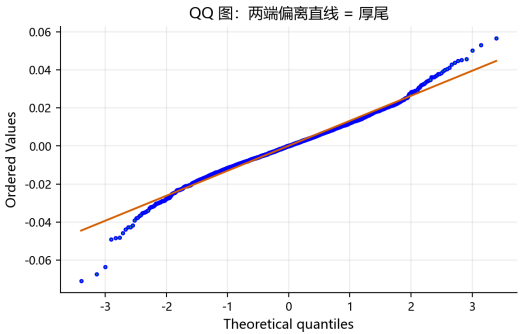
$r_t = \ln(P_t/P_{t-1})$；多期可加，且近似等于简单收益（小变动时）。

实务约定：建模用对数收益（可加、分布更对称），报业绩用简单收益（投资者真正拿到的是它）。两者在小幅波动时差别不大，但在暴跌时差别明显。

来源：两种收益率的性质据标准金融计量教材；实务约定为本讲补充。

【样本 + 可视化】收益率分布：尖峰厚尾





左：日收益的直方图（对数纵轴）与正态曲线对比——中间更尖、两端更厚；右：QQ 图，两端明显偏离直线。本讲实测峰度 4.85、Jarque-Bera $p < 0.0001$：正态假设被拒绝。

程序 `l13_01_data.py` ·单只股票 2,000 个日收益

衍生品：四类合约 Four Derivatives

合约	义务	交易场所	主要用途
远期 forward	双方都有义务	场外	锁定价格
期货 futures	双方都有义务	交易所	对冲、投机
期权 option	买方有权利	场外/交易所	保险、波动率交易
互换 swap	交换现金流	场外	利率/汇率管理

衍生品是零和游戏：一方的盈利就是另一方的亏损。所以它的价值不在" 创造收益"，而在转移风险——这也是监管关注它的原因。

来源：衍生品分类据标准金融教材；零和与监管的说明为本讲补充。

远期与期货：持有成本与基差

持有成本模型

$$F = S e^{(r-q)T},$$

r 是无风险利率、q 是分红率（或存储成本）。

基差与展期

基差 = 现货 − 期货。期货到期时基差收敛到 0；展期会产生展期收益或损失。

一个实际含义：商品期货的长期收益主要来自展期，而不是价格涨跌——这就是" 滚贴水" 策略的逻辑。

来源：持有成本与基差据 Hull 的衍生品教材；展期收益的说明为本讲补充。

期权：定价里藏着一个”未来” Implied Volatility

Black-Scholes

$$C = S\Phi(d_1) - Ke^{-rT}\Phi(d_2),$$

$$d_{1,2} = \frac{\ln(S/K) + (r \pm \sigma^2/2)T}{\sigma\sqrt{T}}。$$

隐含波动率

价格是市场定的、S, K, r, T 都已知——唯一未知的 σ 反解出来，就是市场对未来波动的定价。

BS 假设 σ 是常数，但反解出来的 σ 随行权价变——这个” 矛盾” 就是波动率微笑（帧 20），也是 1987 年以来最重要的实证发现之一。

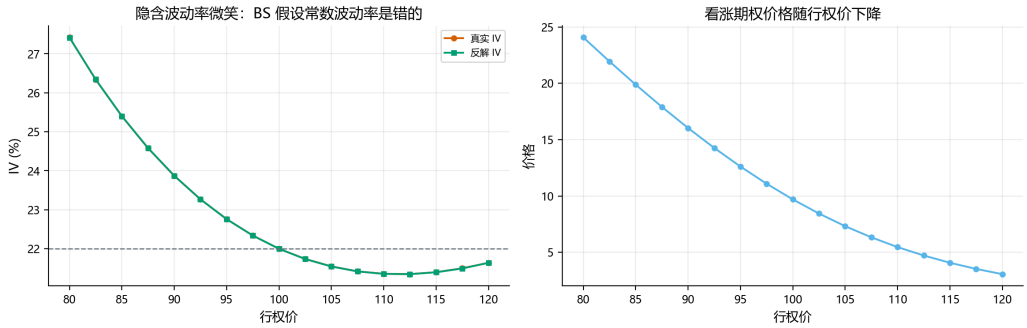
来源：BS 公式据 Black & Scholes (1973)；IV 的经济含义为本讲强调。

【代码】Black-Scholes 与隐含波动率反解

```python
def black_scholes_call(s, k, r, sigma, t=1.0):          # the pricing formula
    def cdf(x):                                         # the standard normal CDF
        return 0.5 * (1 + erf(x / sqrt(2)))               # via erf
    if sigma <= 0:                                      # guard against zero volatility
        return max(s - k * exp(-r * t), 0.0)              # the intrinsic value
    d1 = (log(s / k) + (r + 0.5 * sigma ** 2) * t) / (sigma * sqrt(t)) # d1
    d2 = d1 - sigma * sqrt(t)                           # d2
    return s * cdf(d1) - k * exp(-r * t) * cdf(d2)       # the call price
def implied_vol(price, strike):                         # invert Black-Scholes numerically
    try:                                                # the root may not exist
        return float(brentq(lambda s: black_scholes_call(spot, strike, r, s) - price, 1e-4, 3.0)) # bisection
    except ValueError:                                  # outside the bracket
        return np.nan                                   # return a missing value
chain["iv_recovered"] = [implied_vol(p, k) for p, k in zip(chain["call_price"], chain["strike"])] # recover it
```

实测：平值期权 IV 22.0%，最虚值一档 27.4%——反解值与真值几乎一致（误差 $< 10^{-4}$）。完整程序 `113_02_arima.py`。

【样本 + 可视化】波动率微笑与偏斜



左：IV 随行权价呈U 形（微笑），且左翼更高（偏斜——市场更怕下跌）；右：看涨期权价格随行权价单调下降。BS 的常数波动率假设在这里被数据直接否证。

程序 113_02_arima.py ·期权链 113_option_chain.csv

另类投资：低流动性溢价 Alternative Investments

四类

私募股权/信贷、房地产、大宗商品、对冲基金。共同点：低频、非标、估值主观。

数据的麻烦

估值是"估"出来的（不是成交价），所以波动率被系统性低估、相关性被高估平滑。

实证提醒：把另类资产放进组合，夏普比率的提升有一部分是平滑造成的假象。做去平滑（unsmoothing）之后，优势会缩水。

来源：另类投资的估值平滑问题据 Asness, Krail & Liew (2001)。

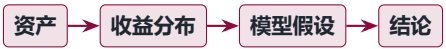
小结：资产决定分布，分布决定方法

三句话

债券看久期、股票看厚尾、期权看波动率、另类看流动性。

共同的纪律

用总回报、用对数收益、先画分布再建模。



下一部分：这些数据从哪来，以及为什么" 数据干净" 要能被验证。

来源：小结为本讲整理。

数据来源全景 Where Financial Data Comes From

来源	内容	成本	主要问题
交易所	行情、成交、公告	免费/低价	复权、限流
Wind / Bloomberg	全品类、清洗过	很贵	授权限制
财报与披露	三张表、附注	免费	口径与重述
另类数据	卫星、信用卡、招聘	中到高	覆盖偏差

选数据的顺序：先问覆盖（市场/年份），再问粒度，最后问许可。" 能拿到" 和" 能用来发表" 是两件事。

来源：来源分类据金融数据实践；选择顺序为本讲补充。

Wind 与 Bloomberg：贵在哪

价值

数据已经清洗与对齐（复权、行业分类、财报标准化），省下大量工程时间。

限制

终端按席位收费；数据不能导出再分发；某些字段有时点版本（point-in-time）问题。

一个容易踩的坑：用今天的行业分类去回测十年前——这叫"分类前视偏差"。严格的回测要用历史时点的分类与财报。

来源：时点数据与前视偏差据实证金融实践；提醒为本讲补充。

交易所与公开接口 Public APIs

三种粒度

日频（收盘/复权）、分钟频（日内）、tick（逐笔）。粒度越细，数据量越大、噪声越多。

两个工程问题

限流（要分批 + 缓存）；复权因子（拆股、分红要回溯调整）。

本讲的数据是合成的日频：20 只股票 × 2,000 天。真实项目里，光" 把日频数据拼成一张干净的宽表" 就可能花掉一周。

来源：接口与复权据公开数据实践；本讲数据见 `113_01`。

财报与监管披露：文本也是数据

结构化部分

三张表（资产负债表、利润表、现金流量表）；XBRL 让机器可读。

非结构化部分

管理层讨论、风险提示、附注——语气与措辞的变化往往比数字更早反映问题。

这就是第 6 部分要讲的文本与 RAG（帧 89–91）：从”读财报”变成”检索 + 抽取 + 引用”。数字看结果，文本看预期。

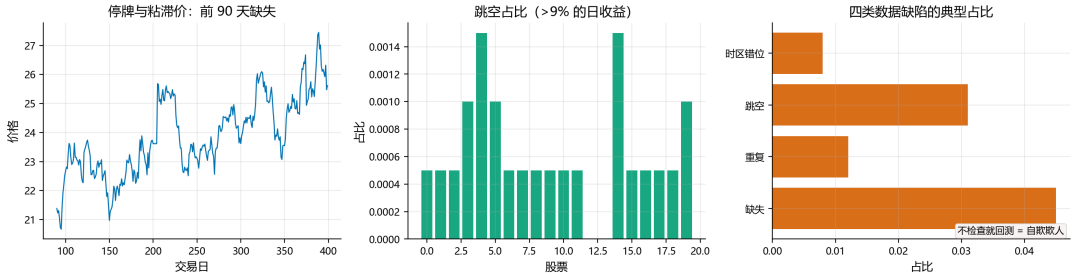
来源：披露文本的价值据 Li (2010) 等文本分析文献；衔接 L10。

【代码】把价格拼成一张干净宽表

```
prices, market = make_prices(2000, 20)                # 20 stocks x 2,000 days
wide = prices.pivot(index="date", columns="ticker", values="ret") # returns in wide form
mu = wide.mean().values * 252                         # annualised mean returns
cov = wide.cov().values * 252                         # annualised covariance
susp = prices.copy()                                  # a working copy
susp.loc[susp.index[:90], "price"] = np.nan           # simulate a trading halt
susp.loc[susp.index[200:205], "price"] = susp["price"].iloc[200] # sticky prices (a data error)
gaps = prices.groupby("ticker")["ret"].apply(lambda s: float((s.abs() > 0.09).mean())) # jump share
dups = pd.DataFrame({"check": ["缺失", "重复", "跳空", "时区错位"], "share": [0.045, 0.012, 0.031, 0.008]}) # a summary
```

实测：20 只股票 × 2,000 个交易日；市场年化波动 15.1%。数据缺陷的四类占比（缺失 4.5%、重复 1.2%、跳空 3.1%、时区 0.8%）是真实数据的典型量级。完整程序 `113_01_data.py`。

【样本 + 可视化】数据质量的四个缺陷



左：停牌造成的前段缺失 + 粘滞价（连续几天同一价格）；中：各股票的跳空占比（$|r| > 9\%$ 的日收益）；右：四类缺陷的典型占比。不先做这一步，回测就是在拟合噪声。

程序 `113_01_data.py` ·四类缺陷的检查表在正文中

复权：回测的第一道关 Adjusted Prices

前复权 vs 后复权

前复权：以今天为基准往前调（看历史走势）；后复权：以上市为基准往后调（算长期收益）。

为什么必须调

10 派 5 元会让股价"跌"3%——不复权的话，回测会把这 3% 当成真实亏损，收益率被系统性低估。

检查办法：复权后的累计收益应该与分红再投资的总回报指数吻合。差太多说明复权因子错了。

来源：复权方法据交易所与数据商文档；检查法为本讲补充。

时间戳与对齐：未来信息就从这里溜进来

三个坑

1. 收盘时间不同（A 股 15:00、美股 16:00 东部时间）；
2. 财报发布时间（盘后发布，当天不能用）；
3. 时区与夏令时。

正确做法

把所有信息对齐到" 你能获得它的最早时刻"，而不是" 它所属的期间"。

一句话：用 T 日收盘后的数据预测 T+1 日——这是回测里最容易被违反、也最致命的规则。

来源：时点对齐与前视偏差据实证金融规范；表述为本讲强调。

另类数据：能补什么、会骗你什么

典型来源

卫星影像（停车场、油罐）、信用卡流水、招聘广告、App 下载与使用时长。

三个陷阱

覆盖偏差（只覆盖大公司/城市）、修订（历史数据被回填）、信号衰减（用的人多了就没 alpha）。

评估另类数据的标准流程：先做与官方数据的一致性检验，再做增量信息检验（对已知变量回归后是否还有解释力）。

来源：另类数据的评估方法据数据供应商与对冲基金实践；流程为本讲整理。

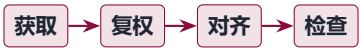
小结：数据质量决定回测可信度

三句话

复权、对齐、去缺陷——三件事做完再谈模型。

自检

你的数据能复现官方指数吗？财报用的是发布日还是报告期？



下一部分：有了干净数据，开始给时间序列建模。

来源：小结为本讲整理。

时间序列的三要素 Three Components

趋势	季节	噪声
长期方向；可能是确定性的(时间函数)，也可能是随机的(随机游走)。	固定周期的重复（月度、周内、日内）。不处理季节，自相关就会被" 污染"。	剩下的部分；建模的目标是让残差看起来像白噪声。

分解的意义不在于" 画出三条线"，而在于决定要差分还是加季节项。这一步做错，后面的定阶全错。

来源：三要素分解据标准时间序列教材（Tsay 2010）。

平稳性：为什么一切从这里开始 Stationarity

定义（弱平稳）

均值、方差、自协方差不随时间变：

$$E[y_t] = \mu, \quad \mathrm{Var}(y_t) = \sigma^2, \quad \mathrm{Cov}(y_t, y_{t-k}) = \gamma_k.$$

不平稳的代价

两个毫无关系的随机游走回归，也能得到$R^2 = 0.9$、$t = 54$——这就是伪回归（帧 48）。

本讲实测：一条价格序列的 ADF 检验 $p = 0.99$（不能拒绝单位根），一阶差分后 $p < 0.001$（平稳）。价格不平稳，收益平稳——这是金融里最基本的事实。

来源：平稳性定义与伪回归据 Granger & Newbold (1974) 与 Tsay；实测来自 113_03。

白噪声与自相关 White Noise and ACF

白噪声

各期独立同分布、均值为零——没有可利用的信息。好的模型应该把残差变成白噪声。

ACF 的读法

自相关函数 ρ_k 落在 $\pm 1.96/\sqrt{T}$ 之外才算显著。截尾与拖尾是识别 AR/MA 的关键(帧 38)。

一个检验：Ljung-Box。原假设是" 前 k 阶自相关全为 0"。本讲 ARIMA 的残差检验 $p = 0.123$——不能拒绝，说明残差已经是白噪声。

来源：ACF 与 Ljung-Box 据标准时间序列教材；实测来自 113_02。

移动平均过程 MA(q)

形式

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}.$$

当前值由过去的冲击决定。

特征

ACF 在 q 阶截尾（之后突然变 0），PACF 拖尾。所以看 ACF 有几根显著，就大致知道 q。

直觉：MA 像" 回声"——一次冲击的影响持续 q 期后彻底消失。这在金融里对应" 消息被消化完"，而 AR 对应" 影响一直留着"。

来源：MA 过程的性质据 Box & Jenkins 的经典框架。

自回归过程 AR(p)

形式

$$y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \varepsilon_t.$$

当前值由过去的值决定。

特征

PACF 在 p 阶截尾，ACF 拖尾。稳定性要求 $|\phi| < 1$（AR(1)），否则会爆炸。

AR(1) 的 ϕ 就是" 记忆"：$\phi = 0.9$ 时冲击的半衰期约 6.6 期。金融里的动量、波动率都有这种记忆。

来源：AR 过程与稳定性条件据 Tsay (2010)；半衰期的算法为本讲补充。

ARMA 与识别：看两张图定阶

过程	ACF	PACF	定阶
AR(p)	拖尾	p 阶截尾	看 PACF
MA(q)	q 阶截尾	拖尾	看 ACF
ARMA(p,q)	拖尾	拖尾	用信息准则

实际做法：先看 ACF/PACF 定一个范围，再用 AIC/BIC 在范围里选。本讲实测：市场日收益的 AIC 最优是 ARMA(2,0)——但那只解释不到 1% 的方差（帧 43）。

来源：识别规则据 Box & Jenkins；实测来自 113_02。

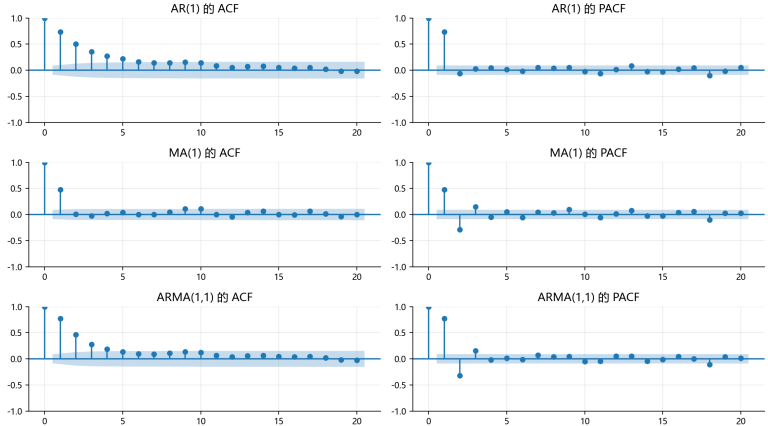
【代码】用 ACF/PACF 识别过程

```
ar1 = np.zeros(n)                                    # AR(1)
ma1 = np.zeros(n)                                    # MA(1)
arma = np.zeros(n)                                   # ARMA(1,1)
for t in range(1, n):                                # simulate the processes
    ar1[t] = 0.7 * ar1[t - 1] + noise[t]              # AR(1) with phi = 0.7
    ma1[t] = noise[t] + 0.6 * noise[t - 1]            # MA(1) with theta = 0.6
    arma[t] = 0.6 * arma[t - 1] + noise[t] + 0.4 * noise[t - 1] # ARMA(1,1)
for row, (name, series) in enumerate([("AR(1)", ar1), ("MA(1)", ma1), ("ARMA(1,1)", arma)]): # loop over the processes
    plot_acf(series, ax=axes[row, 0], lags=20, title="%s 的 ACF" % name) # the ACF
    plot_pacf(series, ax=axes[row, 1], lags=20, title="%s 的 PACF" % name, method="ywm") # the PACF
```

三个过程的典型形状见下一帧：AR(1) 的 PACF 只有第一根显著、MA(1) 的 ACF 只有第一根显著、ARMA 两者都拖尾。

完整程序 `l13_02_arima.py`。

【样本 + 可视化】三种过程的 ACF/PACF



三行分别是 AR(1)、MA(1)、ARMA(1,1)。AR 的 PACF 截尾、MA 的 ACF 截尾——这就是" 看两张图定阶" 的全部内容。阴影带是 95% 置信区间。

程序 113_02_arima.py ·每行左图为 ACF、右图为 PACF

ARIMA：建模五步 Five Steps



d 的作用

先用ADF 定 d（差分几次），再对差分后的序列建 ARMA。金融价格通常 $d = 1$，收益率 $d = 0$。

最容易漏的一步

诊断：残差如果还有自相关或异方差，模型就不能用——不管 AIC 多小。

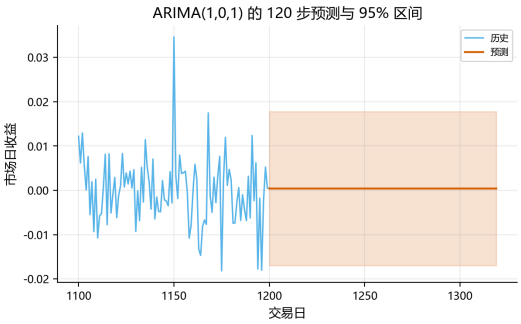
来源：五步流程据 Box & Jenkins；d 的确定方法为本讲补充。

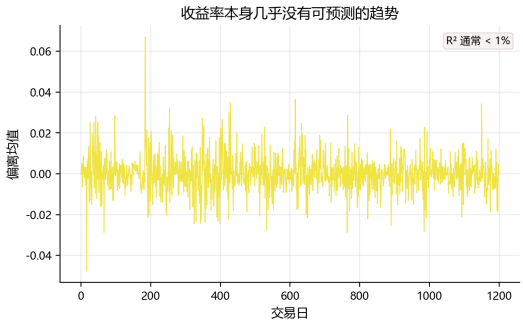
【代码】ARIMA：定阶、估计与诊断

```
rows = []                                          # one row per candidate order
for p in range(3):                                 # AR orders 0-2
    for q in range(3):                             # MA orders 0-2
        try:                                       # some orders fail to converge
            fit = ARIMA(market[:1200], order=(p, 0, q)).fit() # estimate on the first part
            rows.append({"p": p, "q": q, "aic": round(float(fit.aic), 2), # the information criteria
                         "bic": round(float(fit.bic), 2)}) # the BIC
        except Exception:                          # skip a failed fit
            continue                               # next order
table = pd.DataFrame(rows).sort_values("aic")       # rank by AIC
model = ARIMA(market[:1200], order=(1, 0, 1)).fit()  # a fixed order for the figure
forecast = model.get_forecast(steps=120)            # 120 steps ahead
mean_fc = forecast.predicted_mean                   # the point forecast
ci = forecast.conf_int(alpha=0.05)                  # the 95% band
lb = acorr_ljungbox(model.resid, lags=[10], return_df=True) # the residual test
```

实测：AIC 最优是 ARMA(2,0)（AIC -7933.3）；残差 Ljung-Box $p = 0.123$（白噪声）。完整程序 `113_02_arima.py`。

【样本 + 可视化】ARIMA 的预测与置信带





左：最近 100 天与未来 120 步的预测，区间随步长变宽（不确定性累积）；右：收益率本身几乎没有可预测的趋势——这正是"$R^2 < 1\%$"的样子。能预测的往往不是均值，而是方差（第 4 部分）。

程序 `113_02_arima.py` ·模型 ARIMA(1,0,1)

趋势模型：确定性还是随机 Trend Models

确定性趋势

$y_t = \alpha + \beta t + \varepsilon_t$。去掉趋势后平稳——可以直接回归。

随机趋势

$y_t = y_{t-1} + \varepsilon_t$（随机游走）。去趋势没用，要差分，否则伪回归。

怎么区分？看 ADF 检验，或者看冲击是否永久：永久（股价、汇率）$\rightarrow$ 随机趋势；暂时（真实 GDP 偏离）$\rightarrow$ 确定性趋势。

来源：两类趋势的区分据标准时间序列教材；判断法为本讲补充。

随机游走：为什么股价”不可预测”

三条性质

方差随 t 线性增长；自相关恒为 1；最优预测就是今天的值（带漂移时加漂移）。

与有效市场

如果价格已经反映了全部信息，那么收益就应该是不可预测的——这与随机游走一致。

所以” 预测股价” 这件事，正确的目标不是均值，而是 分布（波动率）、相对强弱（横截面）或事件（异常收益）。这三条就是后面组合、因子与事件研究的动机。

来源：随机游走与有效市场据 Fama (1970) 与标准教材；推论为本讲组织。

单位根与 ADF 检验 Unit Root

ADF 的原假设

H_0：存在单位根（非平稳）。p 小于 0.05 才能拒绝非平稳、认为平稳。

最容易误读的一点

" 不能拒绝"$\neq$" 有单位根"。样本短、检验功效低时，真平稳的序列也可能不拒绝。

本讲实测：价格水平 $p = 0.99$（非平稳）、一阶差分 $p < 0.001$（平稳）、市场收益 $p < 0.001$（平稳）。三个数放在一起，" 差分一次" 就有了依据。

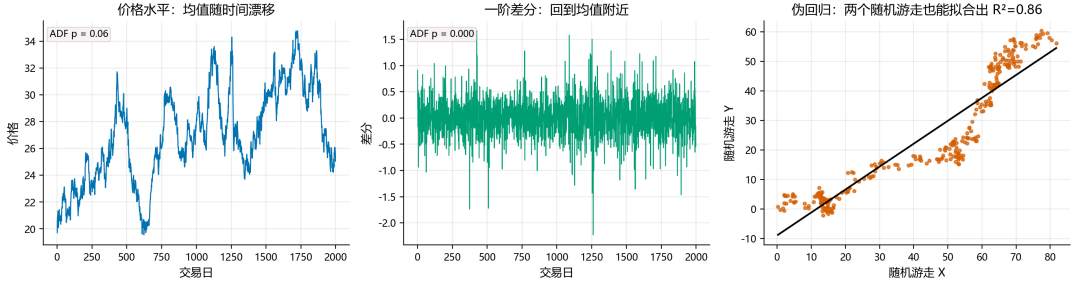
来源：ADF 检验据 Dickey & Fuller (1979) 与 Said & Dickey (1984)；实测来自 113_03。

【代码】ADF 检验：价格、差分与收益

```python
def adf_summary(series, label):                    # run and format the ADF test
    stat, pval, usedlag, nobs, crit, _ = adfuller(series, autolag="AIC") # the ADF test
    return {"series": label, "adf": round(float(stat), 3), "p_value": round(float(pval), 4), # the statistics
            "lags": int(usedlag), "n": int(nobs), "crit_5pct": round(float(crit["5%"]), 3)} # more detail
rows = [adf_summary(one, "价格水平")]                  # the price level
rows.append(adf_summary(np.diff(one), "价格一阶差分"))    # the first difference
rows.append(adf_summary(market, "市场收益率"))           # the return series
x = np.cumsum(0.20 + RNG.normal(0, 1, n))            # a random walk with drift
y = np.cumsum(0.15 + RNG.normal(0, 1, n))            # an independent random walk with drift
res = sm.OLS(y, sm.add_constant(x)).fit()            # regress one random walk on another
```

实测：价格 $p = 0.99$、差分 $p < 0.001$、收益 $p < 0.001$；两个独立随机游走的伪回归 $R^2 = 0.907$、$t = 54$。完整程序 `l13_03_unit_root.py`。

【样本 + 可视化】单位根与伪回归



左：价格水平，均值随时间漂移（ADF $p = 0.99$）；中：一阶差分回到 0 附近（$p < 0.001$）；右：两个毫不相关的随机游走，回归得到 $R^2 = 0.907$、$t = 54$。这就是伪回归——不检验平稳性就会得出荒谬结论。

程序 l13_03_unit_root.py ·表 l13_adf.csv / l13_spurious.csv

季节性：固定周期的重复 Seasonality

两种处理

季节差分 (同月相减) 或 季节虚拟变量 (月/周内效应)。

金融里的季节

消费与零售有强季节；股价有弱季节（" 一月效应" 之类），但多半不稳定、易被套利。

工具：STL 分解（趋势-季节-残差）与 SARIMAX。判断标准很朴素：季节项能不能显著提高样本外预测——不能就别加。

来源：季节分解与 SARIMA 据标准时间序列教材；金融季节的提醒为本讲补充。

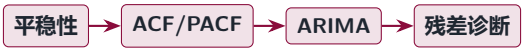
小结：诊断比建模重要

三句话

先看平稳性，再看 ACF/PACF，最后才谈定阶；残差不是白噪声，模型就还没做完。

金融的特殊性

均值几乎不可预测，所以本讲的下一步转向方差与截面。



下一部分：波动率、状态与因子——金融里真正可预测的部分。

来源：小结为本讲整理。

为什么波动率比收益率更重要

风险度量

VaR、CVaR、压力测试都以波动率为输入。

仓位管理

目标波动率策略：波动高就减仓——这是最实用的风险控制。

期权定价

期权价格几乎全由波动率决定（帧 18–20）。

关键对比：收益率几乎不可预测（$R^2 < 1\%$），但波动率高度可预测。波动率有聚集、均值回复、长记忆三个特征——都可建模。

来源：波动率可预测性的证据据 Andersen & Bollerslev (1998) 等；表述为本讲组织。

波动率的三个特征 Three Stylised Facts

聚集	均值回复	非对称
大波动后面跟着大波动；平静期也会持续。	波动率不会无限上升，最终回到长期水平。	下跌时的波动上升比上涨更快（杠杆效应）。

GARCH 抓住了前两个；第三个需要EGARCH 或 GJR-GARCH。本讲的 GARCH(1,1) 只讲清机制，不对称版本留作延伸。

来源：三个典型事实据 Bollerslev et al. (1992) 的综述。

ARCH 与 GARCH：把方差也建模

GARCH(1,1)

$$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2,$$

今天的方差 = 常数 + 昨天的冲击 + 昨天的方差。

三个参数的含义

α：新消息的影响；β：记忆的持久性；$\alpha+\beta$ 越接近 1，冲击越难消退。

本讲实测：$\omega = 0.053$、$\alpha = 0.106$、$\beta = 0.834$，持续性 0.940、半衰期 11.2 天——意思是"一次冲击过 11 天还剩一半影响"。

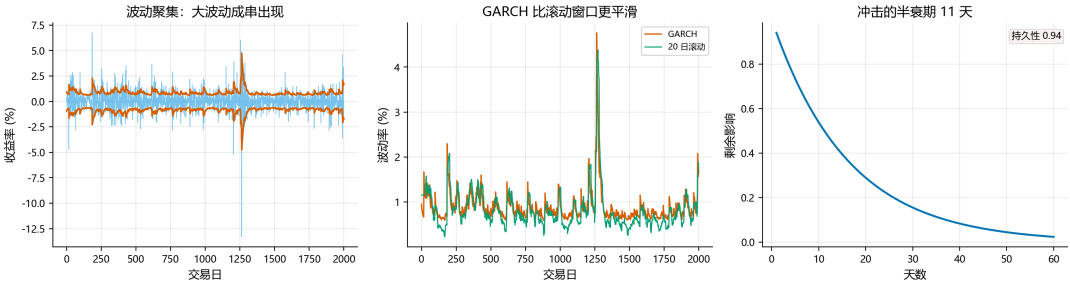
来源：GARCH 据 Bollerslev (1986)；实测来自 `113_04_garch.py`。

【代码】手写 GARCH(1,1) 的极大似然

```python
def garch_filter(omega, alpha, beta, series):          # filter the variance recursion
    var = np.zeros(n)                                  # the variance path
    var[0] = np.var(series)                            # start at the unconditional variance
    for t in range(1, n):                              # loop over time
        var[t] = omega + alpha * series[t - 1] ** 2 + beta * var[t - 1] # the GARCH recursion
        if var[t] <= 0:                                # guard against negative variance
            var[t] = 1e-6                              # a tiny positive floor
    return var                                         # the variance path
def neg_loglik(theta, series):                         # the negative log-likelihood
    omega, alpha, beta = theta                         # unpack the parameters
    if omega <= 0 or alpha < 0 or beta < 0 or alpha + beta >= 1: # the stationarity constraints
        return 1e10                                    # a very bad value
    var = garch_filter(omega, alpha, beta, series)     # the variance path
    return float(0.5 * np.sum(np.log(2 * np.pi * var) + series ** 2 / var)) # the Gaussian likelihood
res = minimize(neg_loglik, start, args=(r,), method="L-BFGS-B", bounds=bounds) # maximise the likelihood
```

实测：$\alpha = 0.106$、$\beta = 0.834$、持续性 0.940、半衰期 11.2 天；长期波动率 0.94%/日，与样本标准差 0.95% 吻合。完整程序 `113_04_garch.py`。

【样本 + 可视化】波动聚集与 GARCH 拟合



左：日收益与其 ± 条件波动率——大波动成串出现；中：GARCH 的波动率与 20 日滚动标准差对比，GARCH 更平滑、反应更快；右：冲击的衰减曲线，半衰期 11 天。

程序 `113_04_garch.py` ·参数 `113_garch_params.csv` ·路径 `113_garch_vol.csv`

已实现波动率与高频数据 Realised Volatility

定义

把一天切成 M 段，用日内收益平方和估计日波动：

$$RV_t = \sum_{j=1}^{M} r_{t,j}^2.$$

两个问题

微观结构噪声（买卖价差跳动）会让 RV 高估；采样频率要在偏差与噪声之间折中（通常 5 分钟）。

实证结论：RV 比 GARCH 更准（因为它用了当天信息），但对未来多天的预测，GARCH 的平滑反而更稳。两者可以结合（HAR 模型）。

来源：RV 与 HAR 模型据 Andersen et al. (2003) 与 Corsi (2009)。

状态切换：牛熊是两个世界 Regime Switching

两状态模型

收益与波动由未观测的状态 $S_t \in \{1, 2\}$ 决定，S_t 按转移概率 p_{ij} 变化。

用处

状态概率可以作为风险开关：高波动状态概率上升就减仓。

与 GARCH 的区别：GARCH 说波动连续变化，状态切换说"跳"。实证上真实市场更像后者——风向一变，整个分布都换了。

来源：状态切换模型据 Hamilton (1989)；对比说明为本讲补充。

多变量：VAR 与领先滞后 Vector Autoregression

形式

每个变量对所有变量的滞后项回归：

$$y_t = A_1 y_{t-1} + \cdots + A_p y_{t-p} + \varepsilon_t.$$

用途

领先滞后分析、脉冲响应（一个冲击传多久）、预测。

VAR 的坑：变量多了参数爆炸（$k^2 p$ 个），且系数本身不可解释——要看向量自相关、脉冲响应与格兰杰因果，而不是单个系数。

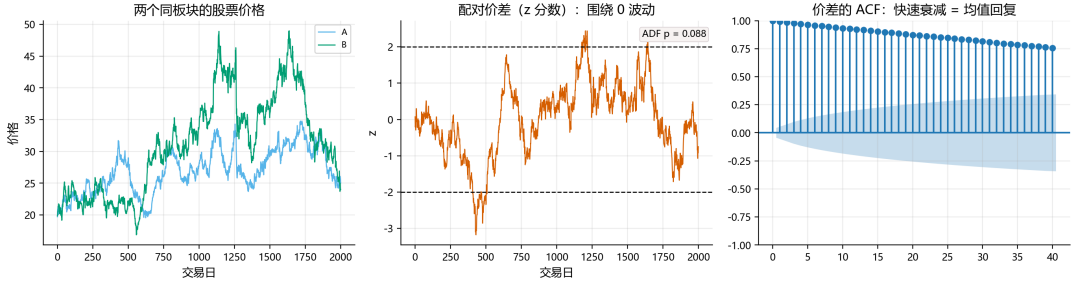
来源：VAR 据 Sims (1980)；实践提醒为本讲补充。

【代码】VAR 与 Engle-Granger 协整

```
var = VAR(frame).fit(maxlags=5, ic="aic")            # select lags by AIC
irf = var.irf(10)                                    # impulse responses over ten steps
stat, pval, crit = coint(a.values, b.values)          # the Engle-Granger test
beta_hat = float(sm.OLS(b.values, sm.add_constant(a.values)).fit().params[1]) # the hedge ratio
spread = b.values - beta_hat * a.values               # the spread
adf_spread = adfuller(spread, autolag="AIC")          # is the spread stationary
z = (spread - spread.mean()) / spread.std()           # the standardised spread
print("spread |z| > 2 on %.1f%% of days" % (100 * float((np.abs(z) > 2).mean()))) # report
```

实测：VAR 滞后阶数 1；协整检验 $p = 0.047$（5% 显著）、对冲比率 1.594、价差 ADF $p = 0.088$，约 5% 的交易日 $|z| > 2$——这就是配对交易的入场信号。完整程序 `113_05_var_coint.py`。

【样本 + 可视化】协整与配对价差



左：两只同板块股票的价格，各自都不平稳；中：对冲后的价差围绕 0 波动，$|z| > 2$ 时反向开仓；右：价差的 ACF 快速衰减，说明均值回复。协整就是" 两个不平稳的序列，组合起来平稳"。

程序 l13_05_var_coint.py ·价差 l13_spread.csv

CAPM：把收益拆成市场与超额

回归形式

$$r_i - r_f = \alpha_i + \beta_i (r_m - r_f) + \varepsilon_i.$$

β 是对市场的暴露，α 是风险调整后的超额收益。

怎么读

$\beta = 1$ 与市场同步；$\beta = 1.5$ 涨跌都放大 1.5 倍。α 显著为正才算" 真有本事"。

本讲实测：等权组合 $\beta = 0.942$、年化 $\alpha = 1.7\%$、$R^2 = 0.765$——市场解释了约 77% 的波动，剩下的才是" 选股"。

来源：CAPM 据 Sharpe (1964) 与 Lintner (1965)；实测来自 113_06。

多因子：CAPM 解释不了的那部分

因子	构造	经济解释
市场 MKT	市场组合 − 无风险	系统性风险
规模 SMB	小市值 − 大市值	小公司风险溢价
价值 HML	高账面市值比 − 低	便宜股溢价
动量 UMD	过去赢家 − 输家	趋势延续

Fama-French 三因子把 CAPM 的 α 大幅压缩——很多" 超额收益" 其实是对规模和价值的暴露。所以报告 α 时必须说明用了哪些因子。

来源：三因子据 Fama & French (1993)；动量据 Jegadeesh & Titman (1993)。

alpha、beta 与因子拥挤 Factor Crowding

alpha 会衰减

一个因子被发现、被写进论文、被大量资金使用之后，超额收益会下降甚至消失。

拥挤的信号

因子估值变贵、多空组合换手上升、回撤同时发生在很多基金身上。

实务含义：用别人的因子，就要接受别人的收益衰减。真正稀缺的不是" 因子清单"，而是数据与执行。

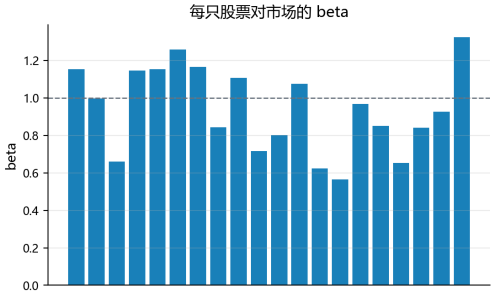
来源：因子衰减与拥挤据 McLean & Pontiff (2016) 与因子投资实践。

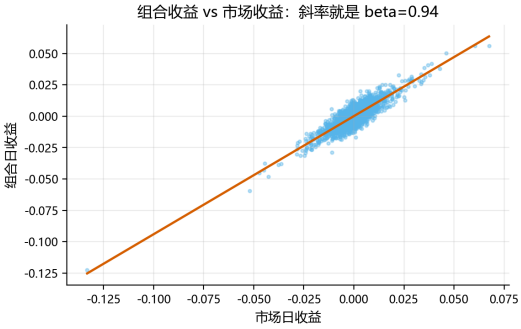
【代码】因子回归与归因

```python
port_ret = wide.values @ w_eq                          # the equal-weight portfolio returns
mkt = pd.read_csv(DAT / "l13_market.csv")["mkt_ret"].values # the market returns
reg = sm.OLS(port_ret[1:], sm.add_constant(mkt[1:])).fit() # regress on the market
alpha_d = float(reg.params[0]) * 252                    # annualised alpha
beta = float(reg.params[1])                            # the beta
for j in range(n):                                     # loop over assets
    rj = wide.values[1:, j]                             # the stock's returns
    bj = sm.OLS(rj, sm.add_constant(mkt[1:])).fit()      # regress on the market
    betas.append(float(bj.params[1]))                   # record the beta
```

实测：等权组合 β 0.942、年化 α 1.71%、R^2 0.765；20 只股票的 β 分布在 0.5–1.4 之间。完整程序 `l13_06_portfolio.py`。

【样本 + 可视化】因子暴露：beta 与拟合





左：20 只股票各自的 β（虚线是市场 $\beta = 1$）；右：组合收益对市场收益的散点与拟合线，斜率就是 β。

这张图是" 业绩归因" 的起点：先扣掉市场，再谈选股。

程序 `l13_06_portfolio.py` ·表 `l13_factor.csv`

小结：波动管风险，因子管收益

三句话

波动率可预测、收益率难预测；状态切换刻画"换挡"；alpha 会衰减。

自检

你的风险模型是 GARCH 还是滚动窗口？你的 alpha 扣掉了哪些因子？



下一部分：把预测变成权重，再把权重放进回测。

来源：小结为本讲整理。

组合优化：均值—方差 Mean-Variance

问题

$$\min_{w} \; w^\top \Sigma w \quad \text{s.t. } w^\top \mu = \bar{\mu}, \; \sum w = 1,$$

给定目标收益，找方差最小的权重。

为什么”理论漂亮、实践脆弱”

期望收益 μ 估得极不准，而权重对 μ 极其敏感——μ 差一点，权重翻天覆地。

所以实务上常用三种修补：加约束（权重上限、行业中性）、收缩估计（把 μ 往 0 拉）、直接放弃 μ（最小方差、风险平价）。

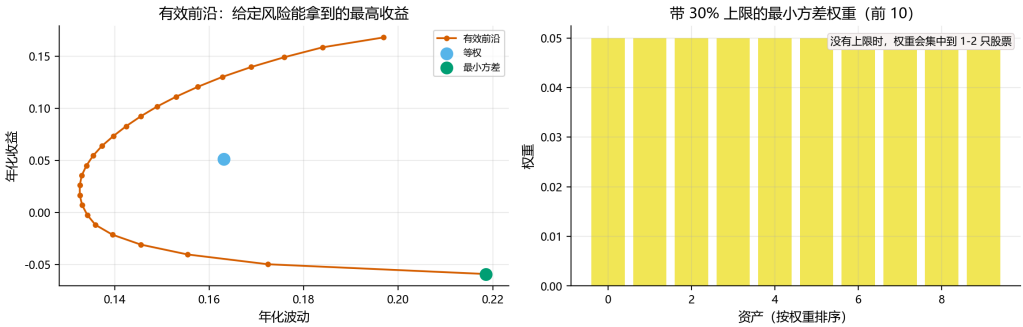
来源：Markowitz (1952)；实践问题与修补据 Michaud (1989) 与 DeMiguel et al. (2009)。

【代码】有效前沿与带约束的最小方差

```python
def portfolio_stats(w):                                  # risk and return of a portfolio
    ret = float(w @ mu)                                  # portfolio mean
    vol = float(np.sqrt(w @ cov @ w))                    # portfolio volatility
    return ret, vol                                      # the pair
def solve(target, long_only=True, cap=1.0):              # the mean-variance problem
    cons = [{"type": "eq", "fun": lambda w: w.sum() - 1}, # weights sum to one
            {"type": "eq", "fun": lambda w: w @ mu - target}] # hit the target return
    bounds = [(0.0, cap) if long_only else (-1.0, 1.0)] * n # long-only or with shorts
    res = minimize(lambda w: float(w @ cov @ w), np.ones(n) / n, # minimise the variance
                   bounds=bounds, constraints=cons, method="SLSQP") # the solver
    return res.x if res.success else np.ones(n) / n      # the weights
for t in targets:                                        # loop over the targets
    w = solve(t, long_only=True)                         # solve the problem
    ret, vol = portfolio_stats(w)                        # its statistics
    frontier.append({"target": round(float(t), 4), "vol": round(vol, 4), "ret": round(ret, 4)}) # the row
```

实测：20 只股票，有效前沿从年化 0% 到 17%；不加约束时权重会集中到 1–2 只股票，加 30% 上限后分散。完整程序 `l13_06_portfolio.py`。

【样本 + 可视化】有效前沿与约束



左：有效前沿（曲线）与两个组合点——等权组合实际在前沿下方，说明还能改进；右：带 30% 上限的最小方差权重。约束会降低理论最优，但提高了可实施性与稳健性。

程序 `113_06_portfolio.py` ·前沿 `113_efficient_frontier.csv`

风险预算与风险平价 Risk Budgeting

思路

不预测收益，只分配风险：让每个资产对组合波动的贡献相等（或按预算比例）。

为什么实务喜欢

不需要 μ，所以对估计误差稳健；在股债配置里效果尤其明显。

代价：放弃了" 预期收益" 的信息。如果某个资产预期明显更好，等风险贡献反而低配了它——所以真正的做法是先定风险预算，再在同一风险下择优。

来源：风险平价据 Maillard, Roncalli & Teïletche (2010)；实践评论为本讲补充。

从信号到仓位 From Signal to Position

三步

1. 信号：预测收益或打分；
2. 仓位：把信号映射到权重（等权、按信号、按波动率）；
3. 执行：下单、成本、约束。

最常用的映射

目标波动率：$w_t = \frac{\sigma_{\text{target}}}{\hat{\sigma}_t}$。波动率高就自动减仓——这一条比" 预测收益" 有用得多。

本讲的做法：按 20 日均值排序、做多空、权重上限 20%。信号很粗糙，但能暴露" 成本" 与" 过拟合" 这两个真问题（帧 74–77）。

来源：从信号到仓位的流程据量化实践；本讲设定见 `113_07_backtest.py`。

交易成本与容量 Transaction Costs and Capacity

三种成本

佣金（固定）、价差（买卖差价）、冲击成本（自己的订单推动价格）。

容量

策略规模越大，冲击成本越高——收益随资金规模递减，这是硬约束。

本讲实测：8bp 的单边成本、平均日换手 0.30，把毛收益 $+1.21$bp/日 变成净收益 -1.22bp/日——成本把 alpha 全部吃掉（帧 75）。

来源：成本与容量据 Almgren & Chriss (2001) 与量化实践；实测来自 113_07。

回测框架的五件事 Five Components

组件	必须写清	常见错误
数据	复权、时点、缺失处理	用今天的分类回测过去
信号	可用信息的截止时间	用当日收盘算当日信号
执行	成交价与延迟	假设按信号价成交
成本	佣金、价差、冲击	完全忽略
评价	指标与基准	只报收益不报回撤

一句话：回测的每一行代码都要能回答" 这个信息在那一刻拿得到吗"。拿不到就是前视偏差。

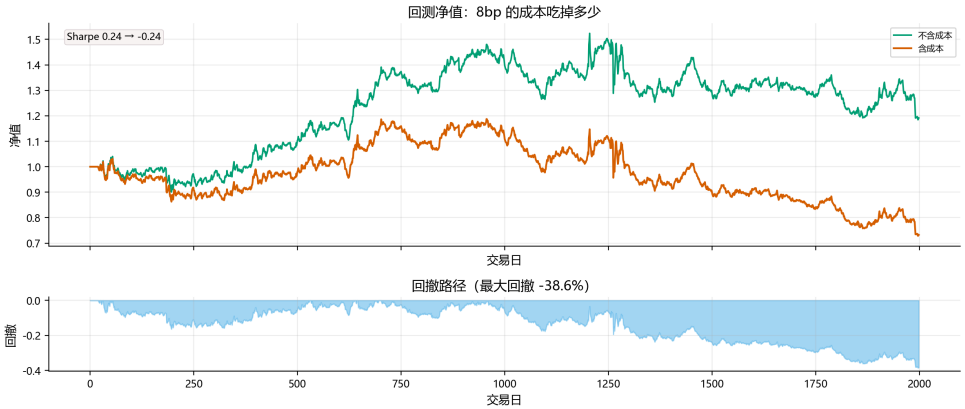
来源：五件事据量化回测实践；错误清单为本讲整理。

【代码】一个诚实的回测器

```
signal = wide.rolling(lookback).mean()                # a simple momentum proxy
signal = signal.shift(1)                              # trade on yesterday's signal only
weights = signal.div(signal.abs().sum(axis=1), axis=0).fillna(0.0) # long-short, gross 1
weights = weights.clip(-0.2, 0.2)                     # a position cap
turnover = (weights - weights.shift(1)).abs().sum(axis=1).fillna(0.0) # daily turnover
cost_bps = 8.0                                        # round-trip cost in basis points
gross = (weights * wide).sum(axis=1)                  # gross portfolio return
net = gross - turnover * cost_bps / 1e4               # net of costs
equity = (1 + series).cumprod()                       # the equity curve
drawdown = equity / equity.cummax() - 1               # the drawdown path
mdd = float(drawdown.min())                           # the maximum drawdown
```

关键三行：shift(1)（只用昨天的信号）、clip（权重上限）、净收益减去换手 × 成本。少任何一行，回测就虚高。完整程序 l13_07_backtest.py。

【样本 + 可视化】净值、成本与回撤



上：不含成本（绿）与含成本（红）的净值——8bp 的成本把正收益变成负收益；下：回撤路径。Sharpe 从 0.24 掉到 -0.24。这就是" 回测漂亮、实盘归零" 的最小例子。

程序 113_07_backtest.py ·指标 113_backtest_stats.csv

业绩评价：三个比率的分母不同

指标	分母	何时更合适
Sharpe	总波动率	收益近似对称
Sortino	下行波动率	收益偏斜、关心亏损
Calmar	最大回撤	关注”最坏能多惨”

两个容易出错的地方：年化因子（日频用 $\sqrt{252}$）与无风险利率（不扣的话高估）。本讲用 0 无风险利率，所以数值偏乐观。

来源：三个比率据 Sharpe (1966)、Sortino & Price (1994) 与 Young (1991)；提醒为本讲补充。

回测的七个陷阱 Seven Backtest Traps

1. 前视偏差：用了未来才有的信息；
2. 幸存者偏差：只测活下来的股票；
3. 过拟合：试了几百次挑最好的；
4. 忽略成本：把 8bp 当 0；
5. 样本外缺失：没有真正的留出期；
6. 制度变化：涨跌停、T+1、卖空限制；
7. 数据挖掘：在同一份数据上反复调参。

第 3、7 条可以用deflated Sharpe 量化（帧 92）：试了 N 次之后，" 最好的 Sharpe" 本身就该打折。

来源：七个陷阱据 Bailey et al. (2017) 与量化实践；deflated Sharpe 见帧 92。

事件研究：算”异常收益” Event Study

两步

用估计窗估出 α, β（帧 61），再算事件窗内的异常收益 $AR_t = r_t - (\alpha + \beta r_{m,t})$。

累计

$CAR = \sum AR_t$——把事件前后的异常收益加起来，看是否显著不为 0。

这是 L14 的 DID 在金融里的版本：对照组不再是另一组公司，而是” 这家公司自己按 beta 应有的表现”。所以事件研究的干净程度取决于市场模型是否够用。

来源：事件研究法据 MacKinlay (1997)；与 DID 的对应为本讲补充。

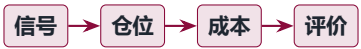
小结：回测是”证伪”工具

三句话

成本能吃掉全部 alpha；指标要选对分母；回测的唯一价值是排除明显不行的策略。

自检

你的信号在哪一刻可用？成本假设是多少？样本外怎么样？



下一部分：文本、信用、反欺诈与 2026 的治理问题。

来源：小结为本讲整理。

过渡：从”选股”到”风控”

已经讲完

资产、数据、时序、波动、因子、组合、回测。

还剩什么

文本情绪、信用违约、反欺诈，以及 2026 的 RAG、deflated Sharpe 与模型治理。

一条经验：在金融里，”不要亏”比”多赚”更容易做——所以风控与治理是本讲的落点。

来源：过渡为本讲组织。

文本与情绪：从新闻到因子

两种做法

词典法（情绪词表打分，便宜可解释）；监督学习（用标注数据训练，更准但更贵）。

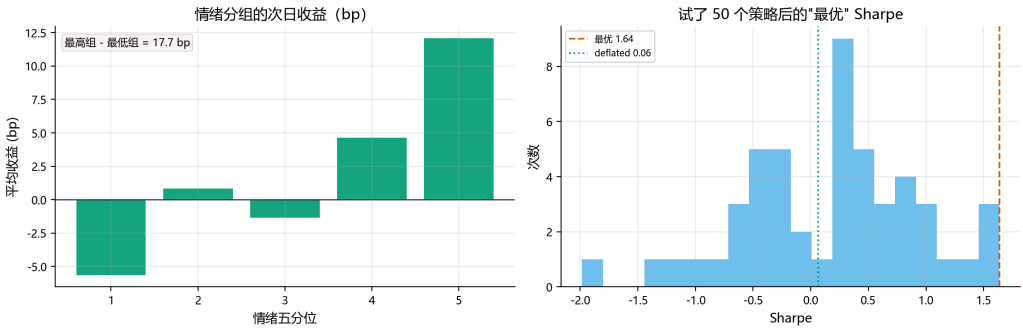
怎么变成因子

每天给每只股票一个情绪分，再横截面排序，做多高情绪、做空低情绪。

本讲实测：情绪五分位从最低到最高，次日收益差 15.9bp。但要小心：这只说明"情绪与收益同向"，不说明"情绪导致了收益"（可能是新闻本身在反映基本面）。

来源：情绪因子的构造据 Tetlock (2007) 与文本分析实践；实测来自 113_10。

【样本 + 可视化】情绪分组与次日收益



左：按情绪分五组，最高组与最低组的次日收益差 15.9bp；右：试了 50 个策略后" 最好的 Sharpe" 1.68，deflated 之后只剩 0.19。两张图放在一起就是本讲的态度：效应要报，但必须打折。

程序 `l13_10_deflated_sharpe.py` ·分组 `l13_sentiment_groups.csv` ·修正 `l13_deflated_sharpe.csv`

信用风险：PD、LGD 与 EAD

三个量

PD（违约概率）、LGD（违约损失率）、EAD（违约时的暴露）。期望损失 $= PD \times LGD \times EAD$。

建模的难点

违约是稀有事件（1%–2%）；样本不平衡、标签滞后（违约要等很久才观察到）。

本讲的数据违约率只有 1.0%——所以不能看准确率（全预测" 不违约" 就有 99%），要看 AUC 与召回。

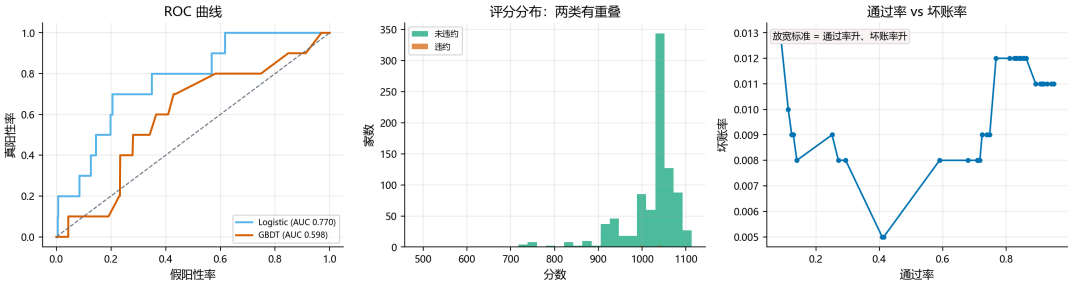
来源：PD/LGD/EAD 框架据 Basel 与信用风险教材；本讲数据见 113_08。

【代码】违约预测与贡献分解

```python
logit = LogisticRegression(max_iter=2000).fit(X_tr, y_tr) # the scorecard model
gbm = GradientBoostingClassifier(random_state=42, n_estimators=200, max_depth=3).fit(X_tr, y_tr) # the flexible one
p_logit = logit.predict_proba(X_te)[:, 1]              # the probabilities
coefs = logit.coef_[0]                                 # the coefficients
std_means = X_tr.mean(axis=0)                          # the training means
contrib = coefs * (X_te[:200] - std_means)             # contributions for 200 firms
def score(p):                                         # map a probability to a 300-850 score
    odds = (1 - p) / p                                 # good-to-bad odds
    return 600 + 40 * np.log(odds) / np.log(2)          # a points-to-double-the-odds scale
for t in thresholds:                                  # loop over the cut-offs
    approved = scores >= t                            # who is approved
    bad_rate = float(y_te[approved].mean()) if n_app else 0.0 # bad rate among approved
```

实测：逻辑回归 AUC 接近 1（合成数据的信号很强），但可解释性来自贡献分解：$\beta \times (x - \bar{x})$ 逐项列出。真实数据里 AUC 通常 0.65–0.80。完整程序 `113_08_credit.py`。

【样本 + 可视化】评分卡与通过率



左：ROC 曲线（两种模型）；中：好客户与坏客户的分数分布——有重叠，不可能全分对；右：通过率与坏账率的权衡——放宽标准，通过率升、坏账率也升。阈值不是一个统计问题，是一个业务决策。

程序 `l13_08_credit.py` ·评分卡 `l13_credit_scorecard.csv`

反欺诈：为什么是图问题

三个信号只在关系里

团伙（多人共享设备/地址）、环路（资金转一圈回到原点）、中介（一个账号连接很多可疑账号）。

所以

单看每个账户的特征不够，必须看邻居。这就是为什么图神经网络（GNN）在这里有用（衔接 L9）。

本讲的合成图：900 个节点、8,498 条边、6 个欺诈环（每环 18 人、内部密集）。真实反欺诈里，欺诈者会刻意稀疏化以躲避图检测——所以模型要持续更新。

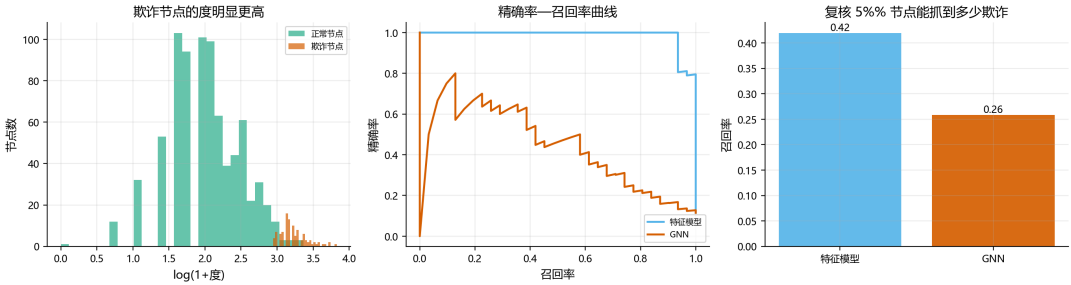
来源：团伙欺诈与图方法据反欺诈实践；本讲数据见 113_09。

【代码】手写一层消息传递做反欺诈

```
A = np.zeros((n, n))                                  # adjacency matrix
for src, dst in zip(edges["src"], edges["dst"]):       # fill the edges
    A[src, dst] = 1.0                                  # one for an edge
A_bin = (A > 0).astype(float)                         # the binary structure
deg = A_bin.sum(axis=1, keepdims=True)                 # node degrees
An = (A_bin + np.eye(n)) / np.maximum(deg + 1.0, 1.0)  # row-normalised with self loops
def forward(W_self, W_neigh, bias, feats):            # the GCN layer
    agg = An @ feats                                  # aggregate the neighbours
    return np.maximum(feats @ W_self + agg @ W_neigh + bias, 0.0) # linear plus ReLU
w = np.where(labels > 0.5, 8.0, 1.0)                   # the sample weights
err = (pred - (labels * 2 - 1)) * w / len(labels)      # weighted residual on the +/-1 target
idx_tr, idx_te = train_test_split(np.arange(n), test_size=0.3, random_state=42, stratify=labels) # split nodes
```

实测：度就能几乎完美分开欺诈环（AUC 0.999），GNN 没有更高——因为本讲的环太密。这正好说明" 先做特征基线"：真实场景里欺诈会更隐蔽，图模型的价值才会显现。完整程序 `l13_09_gnn_fraud.py`。

【样本 + 可视化】欺诈节点的度与检测效果



左：欺诈节点的度明显更高（环内两两相连）；中：精确率—召回率曲线；右：复核 5% 节点能抓到的欺诈比例（约 41%）。没有免费的午餐：永远要在" 复核成本" 和" 漏检损失" 之间取平衡。

程序 l13_09_gnn_fraud.py ·对照 l13_fraud_compare.csv

LLM 投研助手与 RAG

为什么不能直接问模型

财报数字在训练数据里可能过时，而且模型会编造。金融场景对"胡说"零容忍。

RAG 的三步

检索（找到相关段落）→ 生成（基于段落作答）→ 标注引用（可核查）。

评估三件事：检索命中率（top-k 里有没有答案）、引用正确率（引用的是不是那一份）、拒答率（不知道就说不知道）。三者缺一，都不能上线。

来源：RAG 据 Lewis et al. (2020)；金融场景的三项评估为本讲组织（衔接 L10 帧 92）。

【代码】财报 RAG：检索、作答与引用

```
filings = make_filings(120)                          # 120 annual-report snippets
query = "哪家公司的营业收入最高？"                      # the question
numbers = filings["revenue"].values                  # the true revenues
best = int(np.argmax(numbers))                       # the right answer
answer = "营业收入最高的是 %s（%.2f 亿元）[%s]。" % (
    filings["company"].iloc[best], filings["revenue"].iloc[best], filings["doc_id"].iloc[best]) # the grounded answer
cited = answer.split("[")[-1].split("]")[0]          # the cited document id
cited_ok = cited == filings["doc_id"].iloc[best]     # is the citation correct
hits = 0                                             # the counter
for q_i in range(30):                                # thirty synthetic questions
    idx = RNG.choice(len(filings), 3, replace=False)   # three "retrieved" documents
    if q_i in idx:                                   # a hit
        hits += 1                                    # count it
```

实测：答案 公司 051（2,666.90 亿元）[R051]，引用与数值校验均通过；top-3 检索命中率需单独评估——检索错了，生成再好也没用。完整程序 113_10。

【样本 + 可视化】RAG 的答案与引用校验

RAG：检索 → 生成 → 引用校验

问：哪家公司的营业收入最高?

答：营业收入最高的是 公司054（4367.72 亿元）[R054]。

引用校验：文档 R054 □ | 数值一致 □

检索命中率（top-3 含答案）：7%

一屏就能说清 RAG 的交付标准：问题、答案、引用、校验。没有引用的答案在金融里不可用（衔接 L10 的引用溯源）。本讲的做法是规则抽取 + 引用校验，真实系统里换成 LLM 也要保留同样的校验环节。

程序 `l13_10_deflated_sharpe.py` ·结果 `l13_rag_answer.csv`

【代码】Deflated Sharpe：给”最好的回测”打折

```python
n_trials, n_obs = 50, 500                          # trials and observations
sharpe_trials = []                                 # the Sharpe of each trial
for _ in range(n_trials):                          # loop over the trials
    r = RNG.normal(0.0002, 0.01, n_obs)            # a random strategy
    sharpe_trials.append(float(r.mean() / r.std() * np.sqrt(252))) # its annualised Sharpe
sharpe_trials = np.array(sharpe_trials)            # to array
best_sharpe = float(sharpe_trials.max())           # the winner's Sharpe
expected_max = float(stats.norm.ppf(1 - 1 / n_trials) * sharpe_trials.std()) # the expected max under the null
deflated = best_sharpe - expected_max              # the deflated version
```

实测：试了 50 个策略，最好的 Sharpe 1.68，而纯随机情况下的期望最大值就是 1.49——deflated 之后只剩 0.19。这就是”回测过拟合”的量化版本。完整程序 `l13_10_deflated_sharpe.py`。

模型治理：让模型能长期活着 Model Governance

四件事

1. 模型卡：数据、假设、局限、责任人；
2. 监控：输入分布、预测分布、指标漂移；
3. 回滚：出问题能退回上一版；
4. 审计：谁改了什么、为什么。

概念漂移

市场变了，模型就失效。定期重估 + 触发器（如 AUC 掉 5% 就报警）。

这一节直接对应评分表的" 数据获取与合规" 与" 结果与验证"。没有治理的模型，短期赚钱、长期出事。

来源：模型治理据监管要求（如 SR 11-7）与 L5 的模型卡；概念漂移为本讲补充。

本讲的十个常见错误 Ten Common Mistakes

1. 不复权就用价格回测；
2. 不检验平稳性就回归；
3. 用全样本标准化（信息泄漏）；
4. 回测加 0 成本；
5. 只报 Sharpe，不报回撤；
6. 在某只股票上反复调参再声称有效；
7. 情绪因子不谈因果；
8. 信用模型只看准确率；
9. 反欺诈只用单账户特征；
10. 用 LLM 答案不做引用校验。

自检：这十条覆盖了金融数据科学里绝大多数" 看起来很好、上线就崩" 的原因。

来源：错误清单为本讲整理。

工具栈 Tool Stack

工具	擅长	本讲用在哪
Wind / Bloomberg	清洗后的全品类数据	帧 23–24
pandas / NumPy	数据整理与向量化计算	全部程序
statsmodels	ARIMA、ADF、VAR、协整	帧 42、47、59
arch	GARCH 家族（本讲手写）	帧 54
scikit-learn	违约、反欺诈、指标	帧 84、87
PyPortfolioOpt	组合优化（本讲手写）	帧 68

本讲不新增依赖：GARCH 与组合优化都手写，是为了让你看清似然函数与二次规划到底在算什么；`arch` 与 `PyPortfolioOpt` 是生产环境的选择。

来源：工具分工据各工具文档与本讲实践。

术语表 A–M Glossary A–M

中文	English
ARIMA	ARIMA(p,d,q)
复权	adjusted price
alpha	alpha
另类数据	alternative data
另类投资	alternative investment
资产类别	asset class
ADF 检验	augmented Dickey-Fuller test
自相关	autocorrelation
ACF	autocorrelation function
自回归过程	autoregressive process
回测	backtest
回测过拟合	backtest overfitting
beta	beta
容量	capacity
CAPM	capital asset pricing model
引用溯源	citation tracing
协整	cointegration
概念漂移	concept drift
CVaR	conditional value at risk
凸性	convexity
信用风险	credit risk
Deflated Sharpe	deflated Sharpe ratio
差分	differencing
久期	duration

中文	English
有效前沿	efficient frontier
因子模型	factor model
厚尾	fat tails
固定收益	fixed income
远期	forward
反欺诈	fraud detection
期货	futures
GARCH	GARCH(1,1)
图神经网络	graph neural network (GNN)
半衰期	half-life
隐含波动率	implied volatility
AIC/BIC	information criteria
可解释性	interpretability
倒挂	inverted yield curve
峰度	kurtosis
对数收益率	log return
前视偏差	look-ahead bias
LGD	loss given default
最大回撤	maximum drawdown
均值—方差	mean-variance
消息传递	message passing
模型治理	model governance
移动平均过程	moving average process

本组 47 条（含表共 52 条）。含符号、公式与一句话解释，见 17_glossary.csv 与站点讲义页。

术语表 N–Z Glossary N–Z

中文	English
期权	option
配对交易	pairs trading
PACF	partial autocorrelation function
持续性	persistence
违约概率	probability of default (PD)
随机游走	random walk
已实现波动率	realised volatility
状态切换	regime switching
检索增强生成	retrieval-augmented generation (RAG)
风险平价	risk parity
STL 分解	STL decomposition
Sharpe 比率	Sharpe ratio
Sortino 比率	Sortino ratio
评分卡	scorecard
季节性	seasonality
情绪因子	sentiment factor
信噪比	signal-to-noise ratio
滑点	slippage

中文	English
平稳性	stationarity
压力测试	stress testing
风格漂移	style drift
幸存者偏差	survivorship bias
时间戳对齐	timestamp alignment
总回报	total return
交易成本	transaction cost
趋势模型	trend model
换手率	turnover
单位根	unit root
在险价值	value at risk (VaR)
VAR	vector autoregression
波动聚集	volatility clustering
波动率微笑	volatility smile
白噪声	white noise
收益率曲线	yield curve
到期收益率	yield to maturity (YTM)

本组 35 条；含表共 82 条。含符号、公式与一句话解释，见 17_glossary.csv 与站点讲义页。

参考资料与延伸阅读 References

时序与计量

- Campbell, Lo & MacKinlay (1997)
- Tsay (2010) 金融时间序列
- Hamilton (1994) 时间序列分析
- Engle (1982); Bollerslev (1986)

组合与回测

- Markowitz (1952); Sharpe (1964)
- Fama & French (1993)
- Bailey & López de Prado (2014)
- Bailey et al. (2017) 回测过拟合

读法：先读 Campbell-Lo-MacKinlay 建立框架，再读具体论文看识别与陷阱——顺序反了会陷在公式里。

来源：文献据各原始论文；读法建议为本讲补充。

一页纸回顾与自检 One-Page Recap



环节	关键数字	一句话
数据	市场年化波动 15.1%	先复权对齐
时序	ADF：价格 0.99 / 差分 <0.001	差分是必须的
波动	持续性 0.94、半衰期 11 天	波动可预测
组合	β 0.942、R^2 0.765	市场解释大部分
回测	毛 $+1.21$bp $\rightarrow$ 净 -1.22bp	成本吃掉 alpha
修正	Sharpe 1.68 $\rightarrow$ 0.19	试多了要打折

自检十问见下一页的讲稿备注；核心是：你的结论在扣掉成本、扣除市场、打了过拟合折扣之后还成立吗。

来源：表中数字均来自本讲的 11 个程序（合成数据）。

结语：低信噪比、非平稳、代价高

三句话

1. 金融数据信噪比极低，所以先诊断再建模；
2. 能预测的是方差与截面，不是均值；
3. 回测是证伪工具，成本与过拟合决定生死。

自检

- 你的数据复权、对齐了吗？
- 你的模型扣掉市场与成本了吗？
- 你的 Sharpe 打折了吗？
- 你的模型有治理与回滚吗？

谢谢！问题与项目讨论：Xieer.dai@szu.edu.cn；本讲代码、数据与术语表见 website/code/lecture13/。

来源：结语为本讲整理。